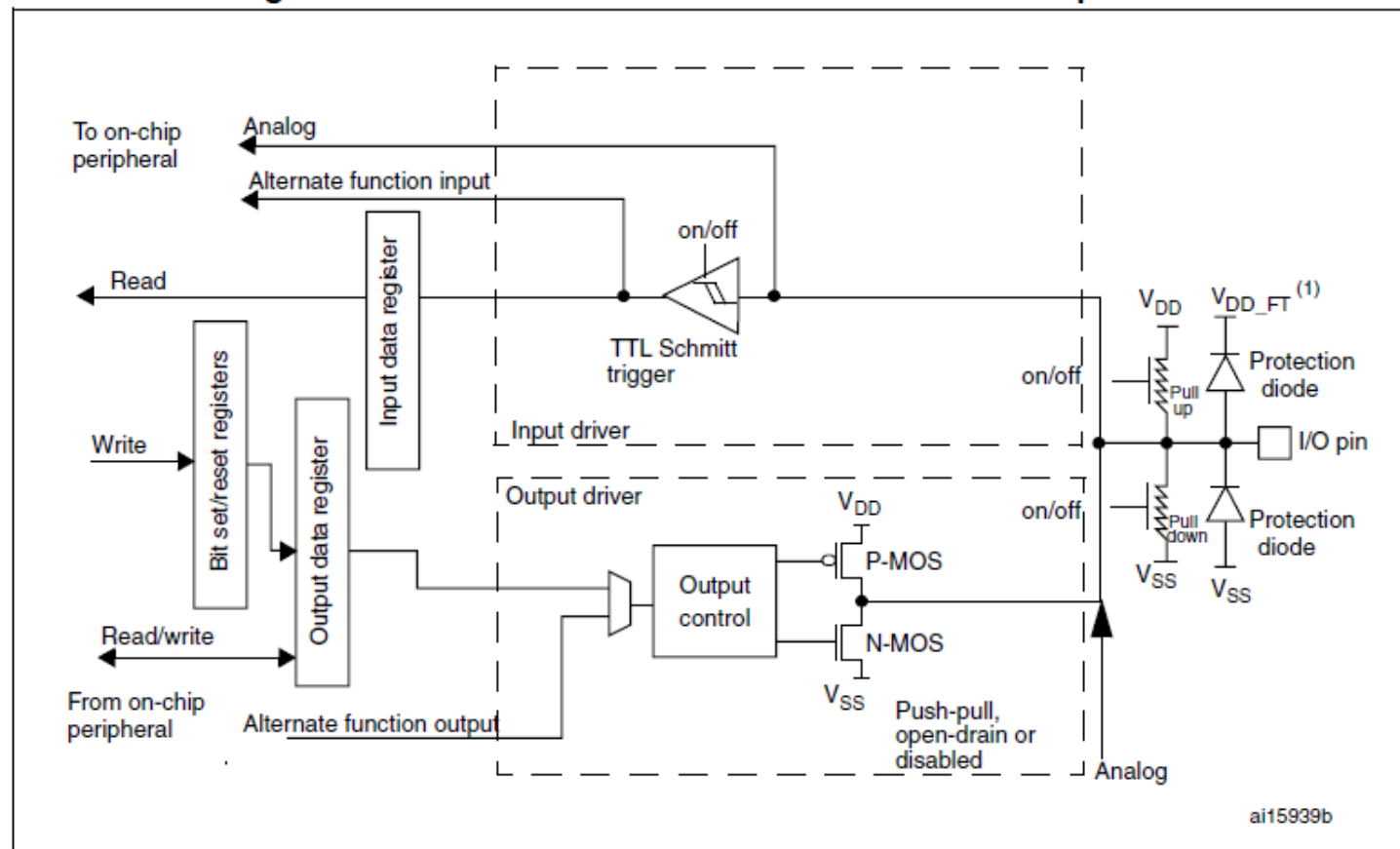


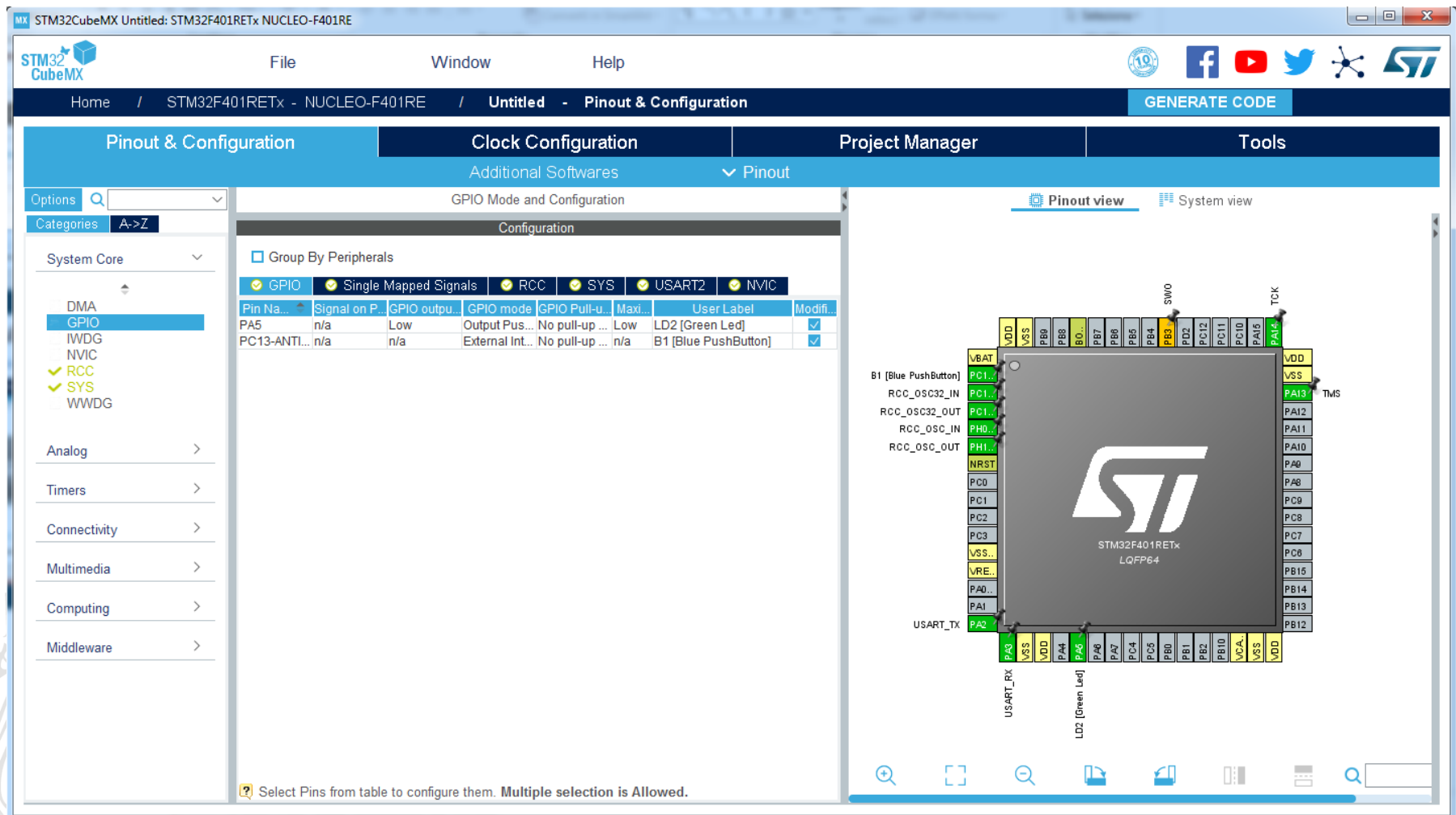
STM32 GPIO (techref)

Figure 16. Basic structure of a five-volt tolerant I/O port bit



1. V_{DD_FT} is a potential specific to five-volt tolerant I/Os and different from V_{DD} .

STM32CubeIDE: GPIO



STM32CubeMX Untitled: STM32F401RETx NUCLEO-F401RE

File Window Help

Home / STM32F401RETx - NUCLEO-F401RE / Untitled - Pinout & Configuration

GENERATE CODE

Pinout & Configuration Clock Configuration Project Manager Tools

Additional Softwares Pinout

Options Search Categories A-Z

System Core

- DMA
- GPIO
- IWDG
- NVIC
- ✓ RCC
- ✓ SYS
- WWDG

Analog Timers Connectivity Multimedia Computing Middleware

Configuration

☐ Group By Peripherals

☒ GPIO ☒ Single Mapped Signals ☒ RCC ☒ SYS ☒ USART2 ☒ NVIC

Pin Na...	Signal on P...	GPIO output...	GPIO mode	GPIO Pull-u...	Maxi...	User Label	Modifi...
PA5	n/a	Low	Output Pus...	No pull-up ...	Low	LD2 [Green Led]	☒
PC13-ANTI...	n/a	n/a	External Int...	No pull-up ...	n/a	B1 [Blue PushButton]	☒

Pinout view System view

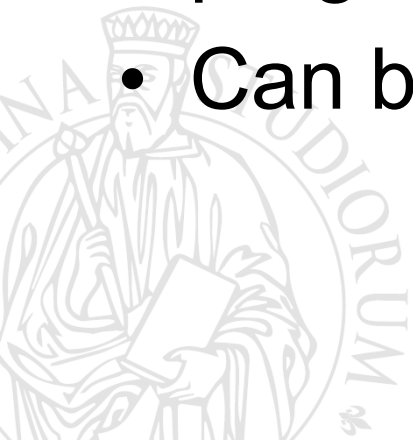
STM32F401RETx LQFP64

Select Pins from table to configure them. Multiple selection is Allowed.



Interrupt Controller

- Can deviate the normal program execution to a different routine, in response to an event
- After execution of the routine (ISR) the program continues execution as before
- Can be Nested/Vectored (NVIC)
(see techref)

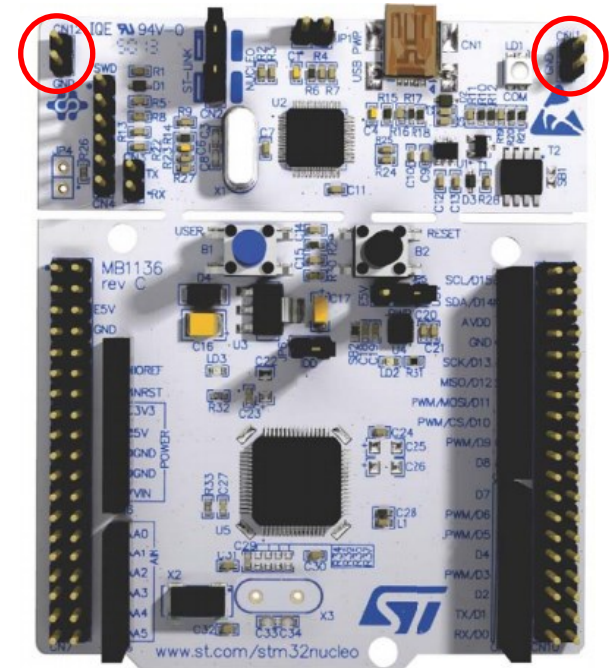


Lab Exercise:

- Start a new project (for F411 Nucleo-64 boards)
- In the While (1) loop: test the following GPIO functions on the LD2 (Green Led pin, **verify looking at the LED**):
 - HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
 - HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
 - Alternate two Write functions with Pinstate = 0 and PinState = 1
- You can also read back the actual Output value using:
 - HAL_GPIO_ReadPin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
- Port name and Pin Number for the LED can be found in main.h definitions

Lab Exercise, directions:

- Check the GPIO connected to the LED and locate the corresponding pin on the expansion headers
- The board has 2 ground headers to attach the oscilloscope probe





Lab Exercise:

- Start a new project
- Rename the Blue Button GPIO and LED GPIO
- Set the Button as simple input GPIO
- Read the Button and toggle the led every time the button is pressed.





Lab Exercise, directions:

- HAL_GPIO_TogglePin can be used, but be careful, this function is not interrupt-safe
- Use HAL_Delay for debouncing the button





Lab Exercise:

- Now, set the Button as EXTI interrupt source and enable the ISR in the NVIC
- Use the ISR to Toggle the LED
- How to debounce now? (i.e. it is not possible to use HAL_Delay in an ISR...)





Lab Exercise, directions:

- You'll need some static variables, to keep the status
- Even if HAL_Delay cannot be used, you can still use HAL_GetTick to mark the first event and to calculate an 'hold-off' period for debounce.





Lab Exercise:

- Let's verify the GPIO HAL libraries:
- In the while (1) loop, using HAL functions, toggle a GPIO output continuously (write 1, write 0, without delays)
- Use the oscilloscope to measure the time needed to perform the said operations.
- Now, take a look into the HAL_GPIO_WritePin function and try to extract only the strictly relevant operations
- Place the stripped operations in the while (1) loop
- Measure again the time needed.